Capítulo 4

Vetores Ordenados – Busca Binária

Exercícios

1.(ENADE): A **busca binária** possui complexidade de tempo diferente da busca sequencial. Suponha um vetor de tamanho n já ordenado. Qual das alternativas descreve **corretamente** a complexidade de tempo da busca binária, em termos de melhor caso e pior caso, comparando-a à busca sequencial?

A. Melhor caso O(n) e pior caso O(n^2) na busca binária, o que é mais eficiente que a busca sequencial.
B. Melhor caso O(1) e pior caso O(log n) na busca binária, enquanto a busca sequencial tem melhor caso O(1) e pior caso O(n).
C. Melhor caso e pior caso O(log n) na busca binária, pois sempre dividimos a busca pela metade; a busca sequencial sempre é O(n) em qualquer caso.
D. Melhor caso O(1) e pior caso O(n) tanto para busca binária quanto para busca sequencial.

2.(TEÓRICA): Por que o algoritmo de **busca binária exige que o vetor esteja ordenado** previamente? O que poderia acontecer se tentássemos aplicar a busca binária em um conjunto de dados não ordenado?

3.(PRÁTICA): Implemente a função `busca_binaria` de forma recursiva em Python. Ou seja, escreva uma versão da busca binária que chame a si própria recursivamente em vez de usar um loop while. A função deve receber como parâmetros o vetor ordenado, o valor alvo buscado, e opcionalmente os índices esq e dir delimitando o segmento de busca (ou você pode definir a função interna aninhada com esses parâmetros). Retorne o índice do alvo no vetor caso seja encontrado, ou -1 caso contrário.

4.(PRÁTICA): Considere uma lista Python de 1000 números inteiros já ordenados em ordem crescente. Escreva um pequeno programa que utilize tanto a busca sequencial quanto a busca binária para encontrar um determinado número nessa lista e *conte o número de comparações* feitas em cada método. Faça o teste com um número que *não* esteja presente na lista (para forçar o pior caso) e exiba na saída o total de comparações realizadas por cada método. *(Dica: você pode modificar as funções de busca para incrementarem um contador global ou utilizar variáveis externas para contar as comparações.)*

Projetos

Para ilustrar a aplicação prática da busca binária no contexto de Ciência de Dados para Negócios, vamos explorar dois projetos:

Projeto 1: Busca Eficiente em um Catálogo de Produtos Online

Contexto: Imagine que você está desenvolvendo a funcionalidade de busca para um site de e-commerce com um grande número de produtos. O catálogo de produtos é armazenado em uma lista, onde cada item é um dicionário contendo informações como nome, preço e descrição. Para otimizar a busca, o catálogo é mantido ordenado alfabeticamente pelo nome do produto.

Problema: Os clientes precisam encontrar rapidamente informações sobre um produto específico digitando seu nome na barra de busca. Uma busca lenta pode levar à frustração e à perda de vendas.

Indicação de como resolver: *Implemente um sistema de busca que utilize a busca binária para localizar o produto no catálogo ordenado. Ao receber o nome do produto digitado pelo usuário, sua função deve realizar a busca binária na lista de produtos. Se o produto for encontrado, retorne suas informações (por exemplo, preço, descrição). Caso contrário, informe que o produto não foi encontrado.*

Projeto 2: Busca Eficiente em um Índice de Livros

Contexto: Você está trabalhando em um projeto para gerenciar uma biblioteca. As informações sobre os livros (título, autor, ano de publicação) estão armazenadas em uma lista, e essa lista é mantida ordenada alfabeticamente pelo título do livro.

Problema: Os usuários precisam encontrar rapidamente um livro específico sabendo seu título. Uma busca ineficiente pode dificultar a localização dos livros desejados.

Indicação de como resolver: *Desenvolva uma funcionalidade que permita aos usuários buscar um livro pelo título utilizando a busca binária. Ao receber o título do livro, sua função deve procurar no índice ordenado. Se o livro for encontrado, exiba suas informações (título, autor, ano). Caso contrário, informe que o livro não foi encontrado.*